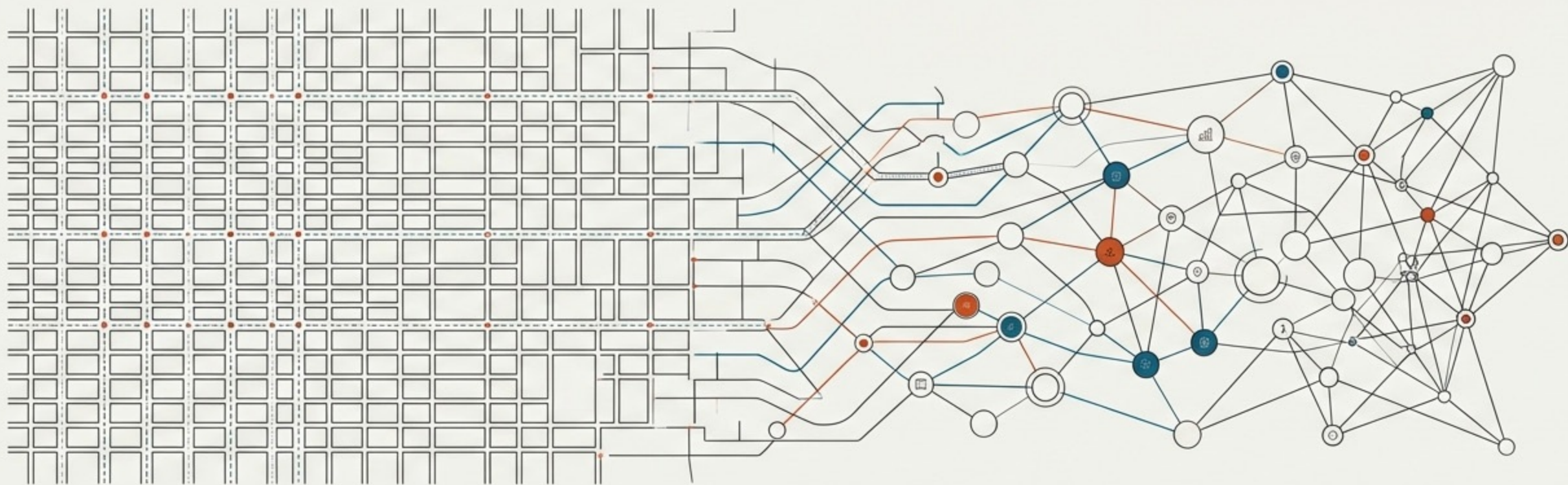


图论算法核心讲义

从城市规划到算法架构的系统化图解



解码图的遗传密码：万物皆可互联

从现实世界到数学模型

什么是点和边？

$$G=(V;E)$$

顶点V:
“具体实体”



边E:
“结构关系”



图论仅关注实体间的结构关系，而非其个体属性。

图的交通规则

方向性（方向）

无向图



有向图



权重（权重）



无权图

Concerned only with connectivity
(e.g., friendship networks)



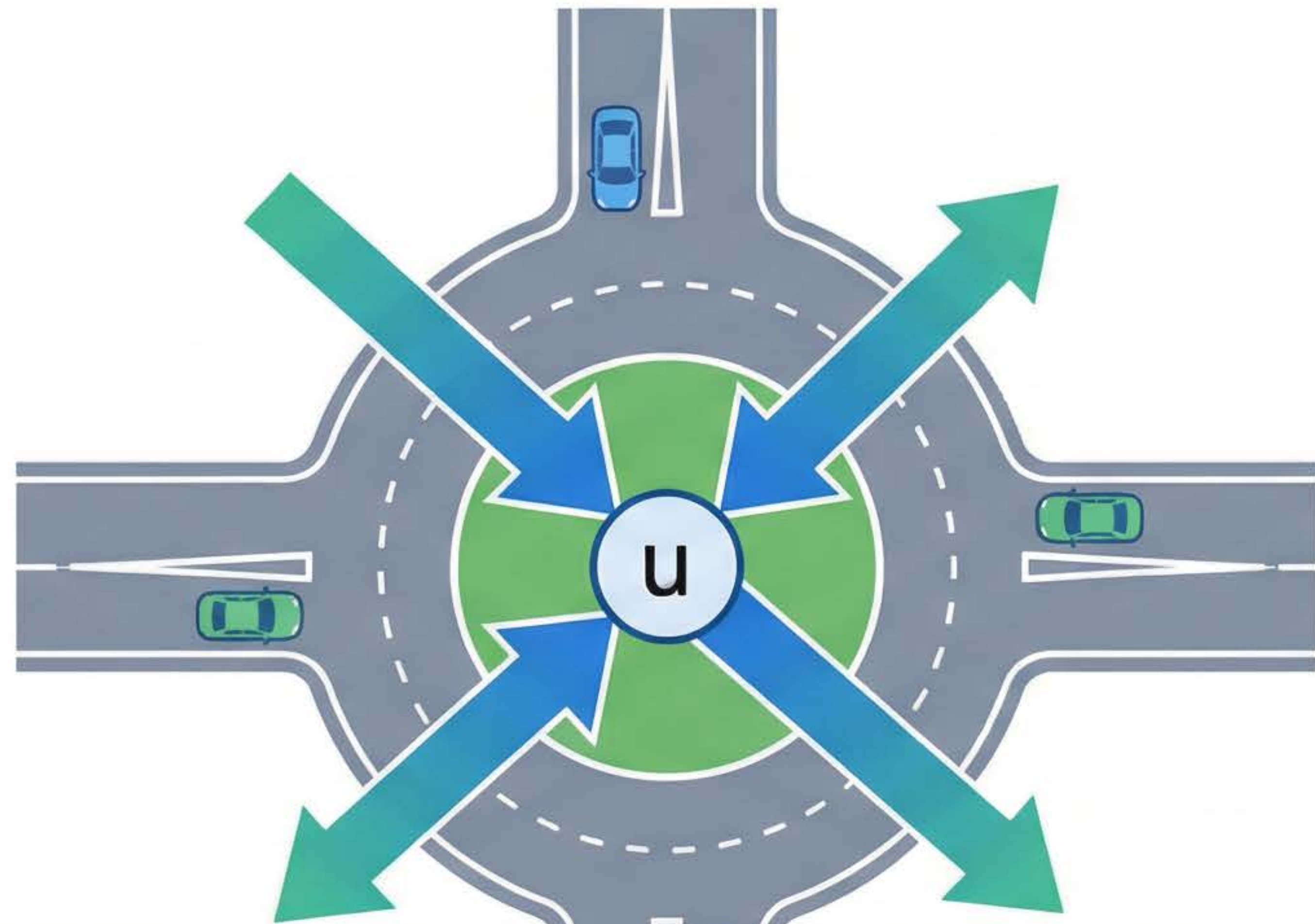
有权图

Concerned with costs or distances

局部枢纽与全局环流

理解度 (Degree) 与环 (Cycle)，是掌握高级算法的钥匙

度数分析 (节点的流量监控)



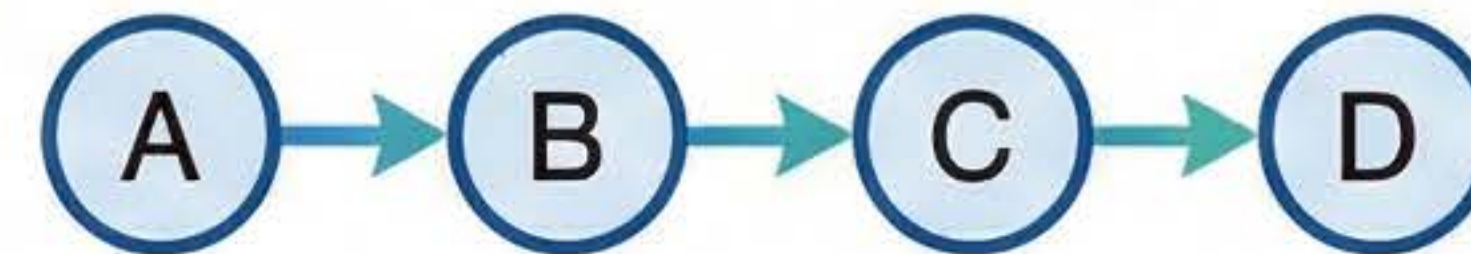
入度 (In-degree) = 3
有多少条路通向我? (决定了拓扑排序中谁能先执行)

出度 (Out-degree) = 2
我能通向多少个地方?
(决定了向外扩散的广度)

环路检测 (无尽的死循环)

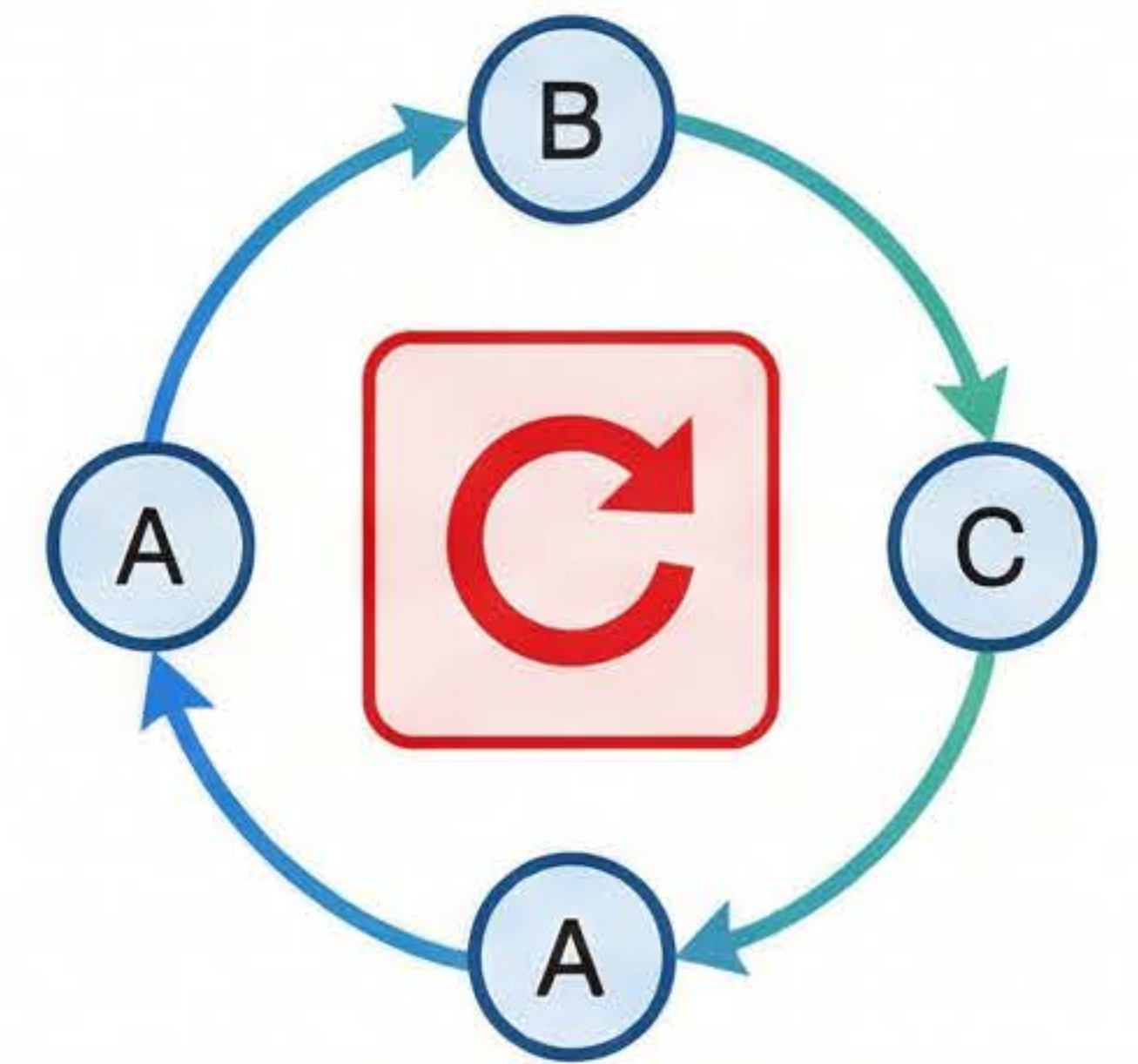
图 A

DAG (有向无环图)



清晰的先后依赖关系,
工程排期的基础。

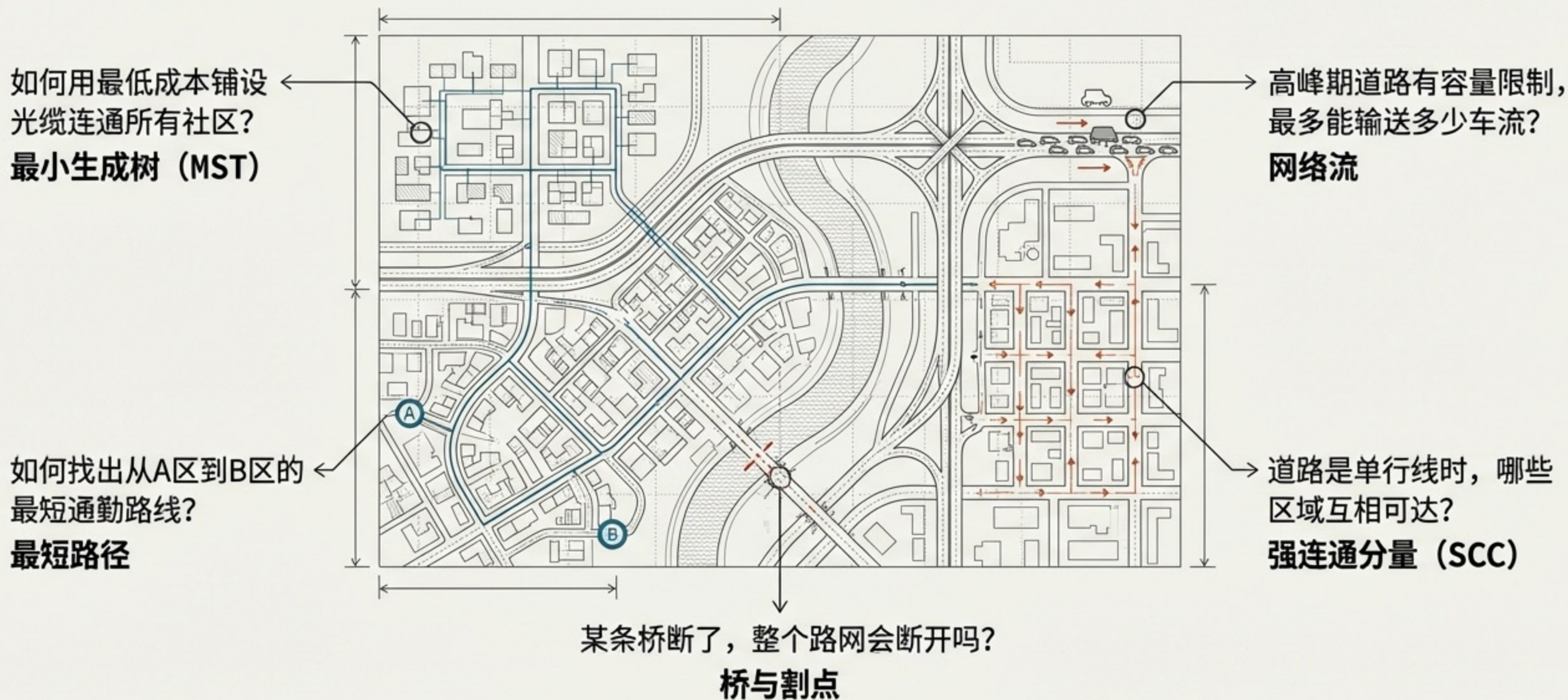
图 B



环 (Cycle)

! 环是图论算法中最容易引发Bug的结构 (如死循环)。后续的 Bellman-Ford 和 Tarjan 算法, 核心使命之一就是发现并处理环。

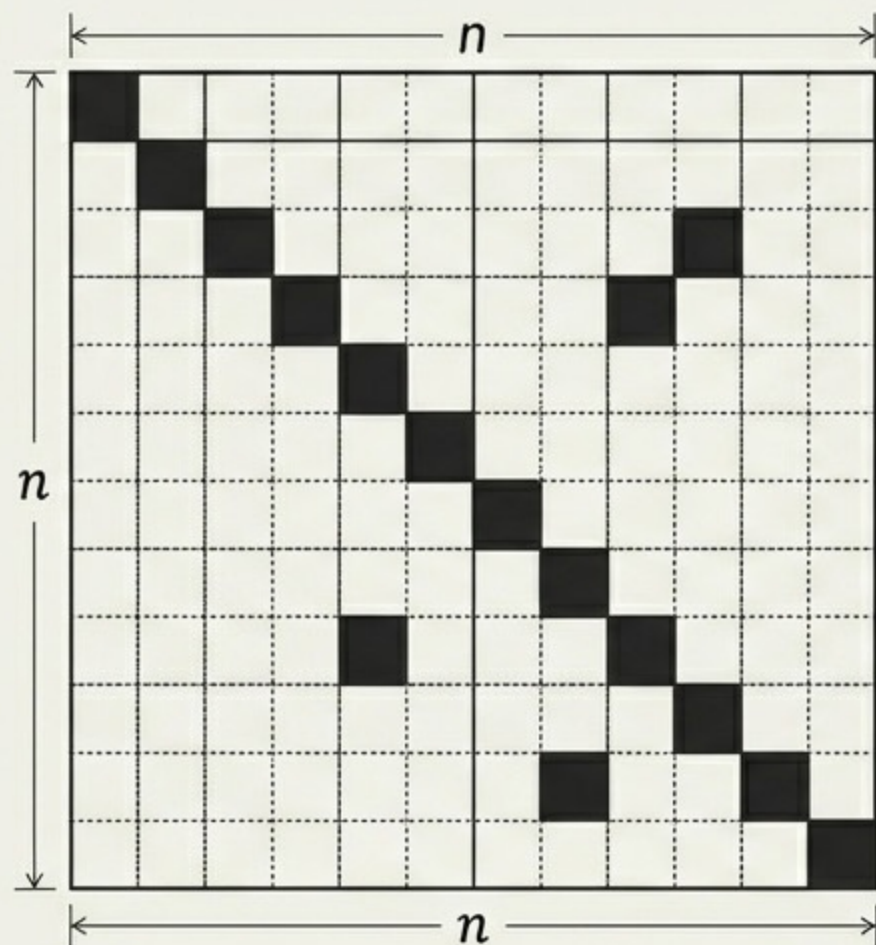
引子：你现在的身份是城市交通规划师



解决这些问题的钥匙, 始终是两种最朴素的行走方式——逐层扩散 (BFS) 与 一路走到底 (DFS)。

图的表示：构建路网蓝图

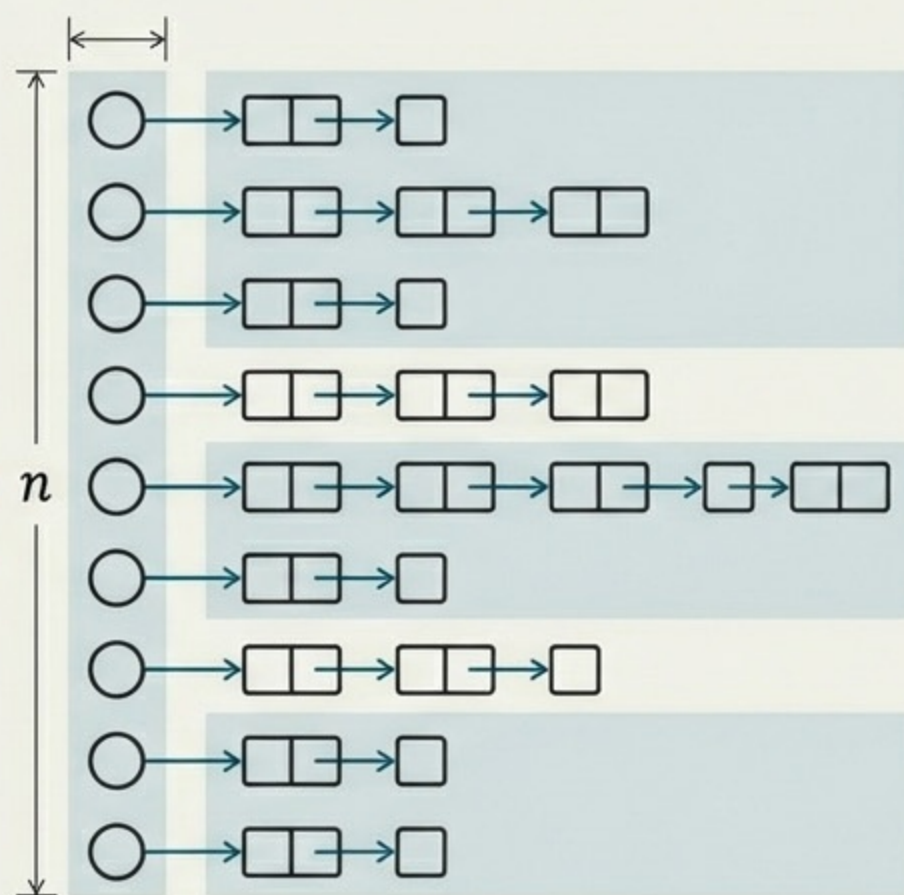
$$G = (V, E), |V| = n, |E| = m$$



邻接矩阵 (Adjacency Matrix)

空间: $O(n^2)$ | 适用: 稠密图

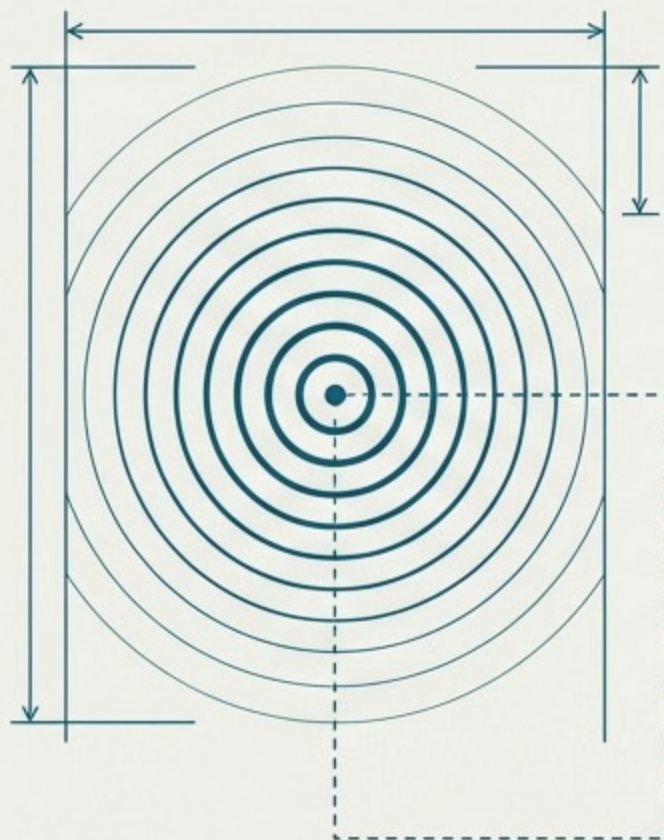
查边效率:
 $O(\text{deg})$ vs $O(1)$



邻接表 (Adjacency List)

空间: $O(n+m)$ | 适用: 大多数场景首选

两种遍历：算法设计的双引擎



BFS (广度优先)

结构：
队列 (Queue)

行为：
先发现的先处理
→ 天然按距离分层

超级能力：
无权图的最短路径
($O(n+m)$)



DFS (深度优先)

结构：
递归/栈 (Stack)

行为：
记录时间戳：
发现($d[v]$) & 完成($f[v]$)

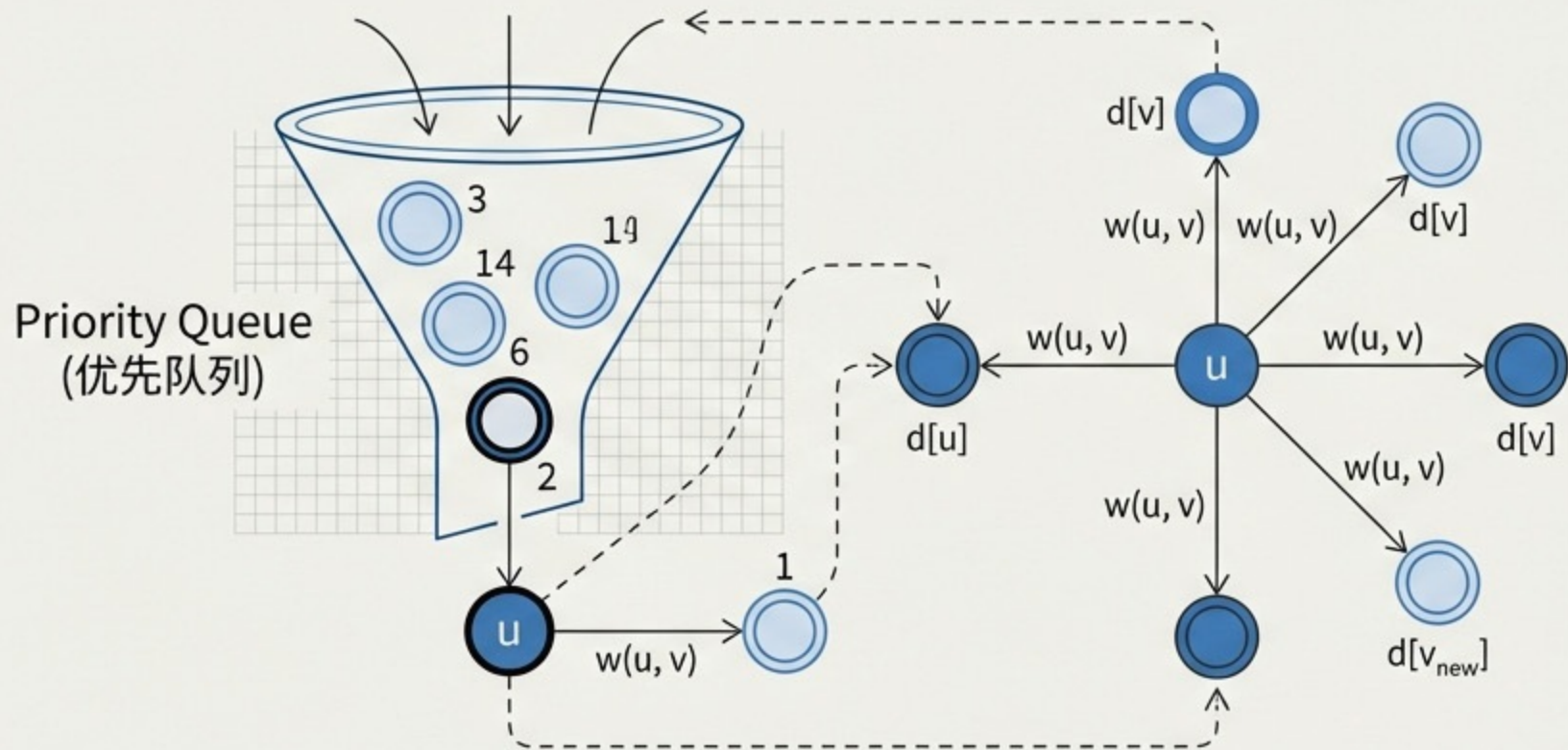
超级能力：
揭示深层结构
(环检测, 拓扑排序, SCC)

Every advanced algorithm uses one of these two patterns.

最短路径 I: 松弛与贪心

$\text{Relax}(u, v): \text{if } d[v] > d[u] + w(u, v) \text{ then } d[v] \leftarrow d[u] + w(u, v)$

算法的区别仅在于松弛的顺序。



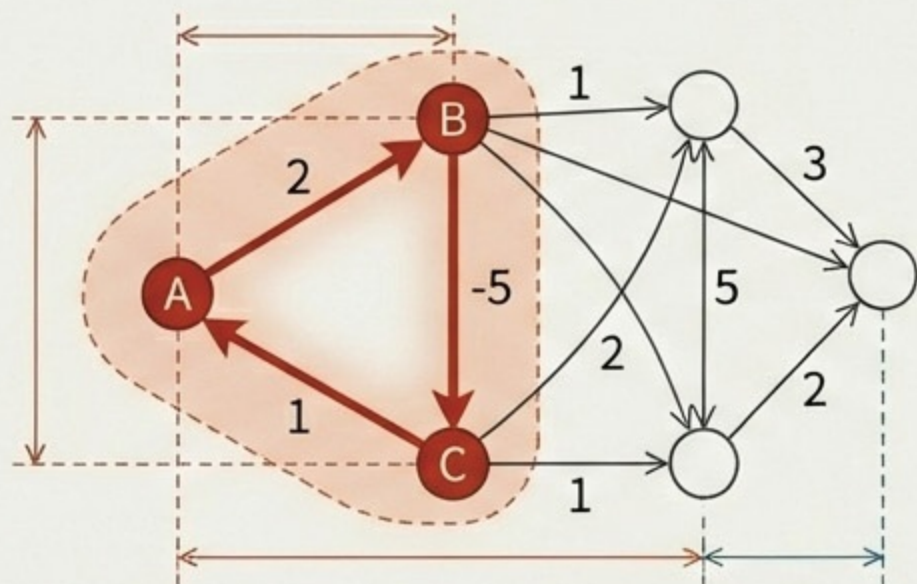
Dijkstra (Greedy BFS)

$O((n + m) \log n)$

✗ 不能处理负权边

最短路径 II: 负权与全局视角

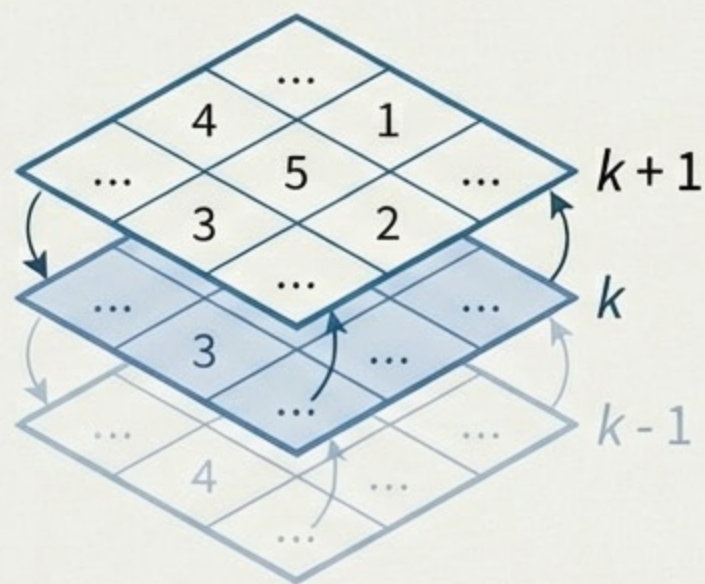
Bellman-Ford



- 暴力松弛 $N-1$ 轮
- ✓ 可检测负权环

$O(nm)$

Floyd-Warshall



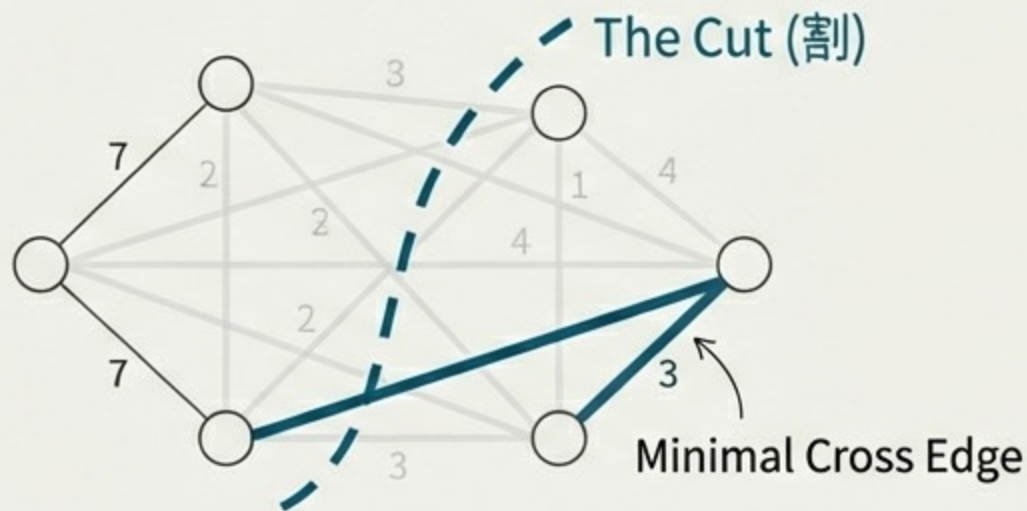
- 上帝视角 (All-Pairs)
- $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$

$O(n^3)$

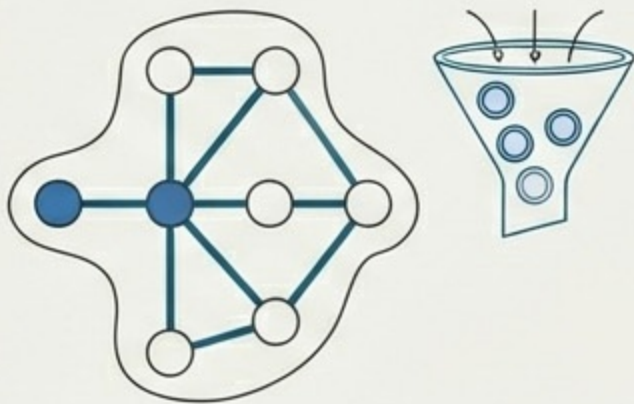
最小生成树 (MST)：低成本连通方案

The Cut Property (割性质)

任意割的最小横跨边必在MST中。

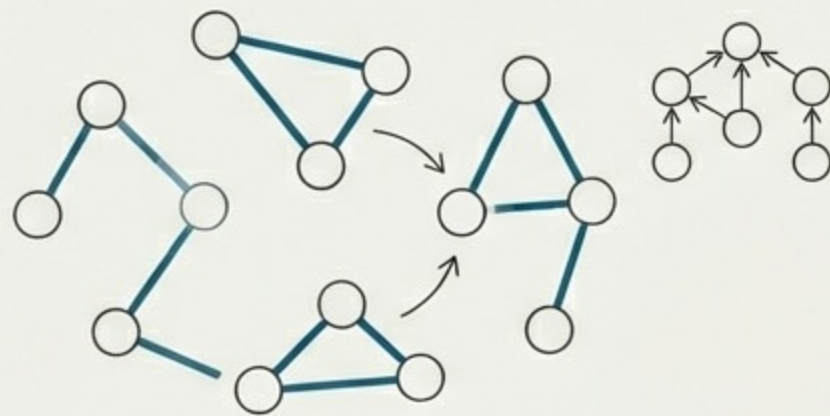


Prim



点优先 (Priority Queue) | 适合稠密图

Kruskal



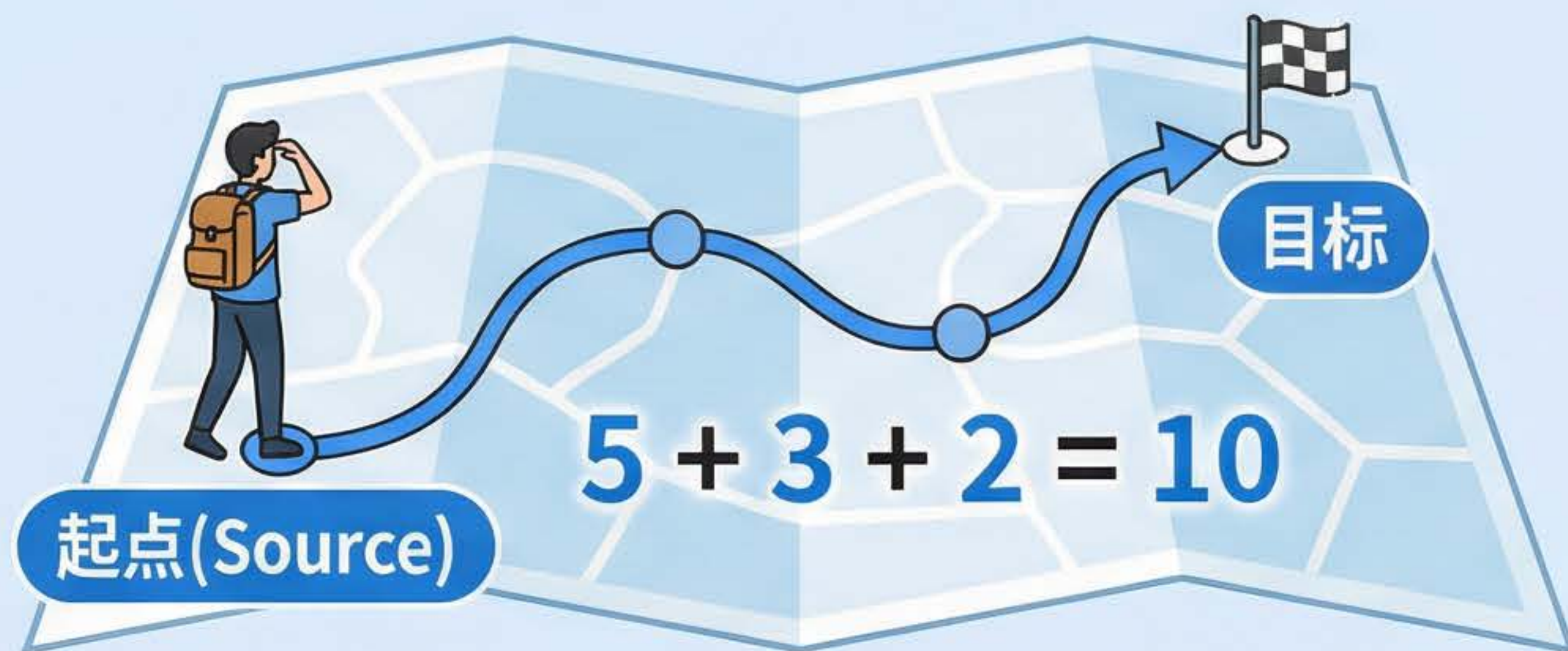
边优先 (Union-Find) | 适合稀疏图

贪心双子星的“世纪对决”

为什么代码看起来一样，解决的问题却天差地别？

Dijkstra（单源最短路径）

🏆 “极致的个人主义”



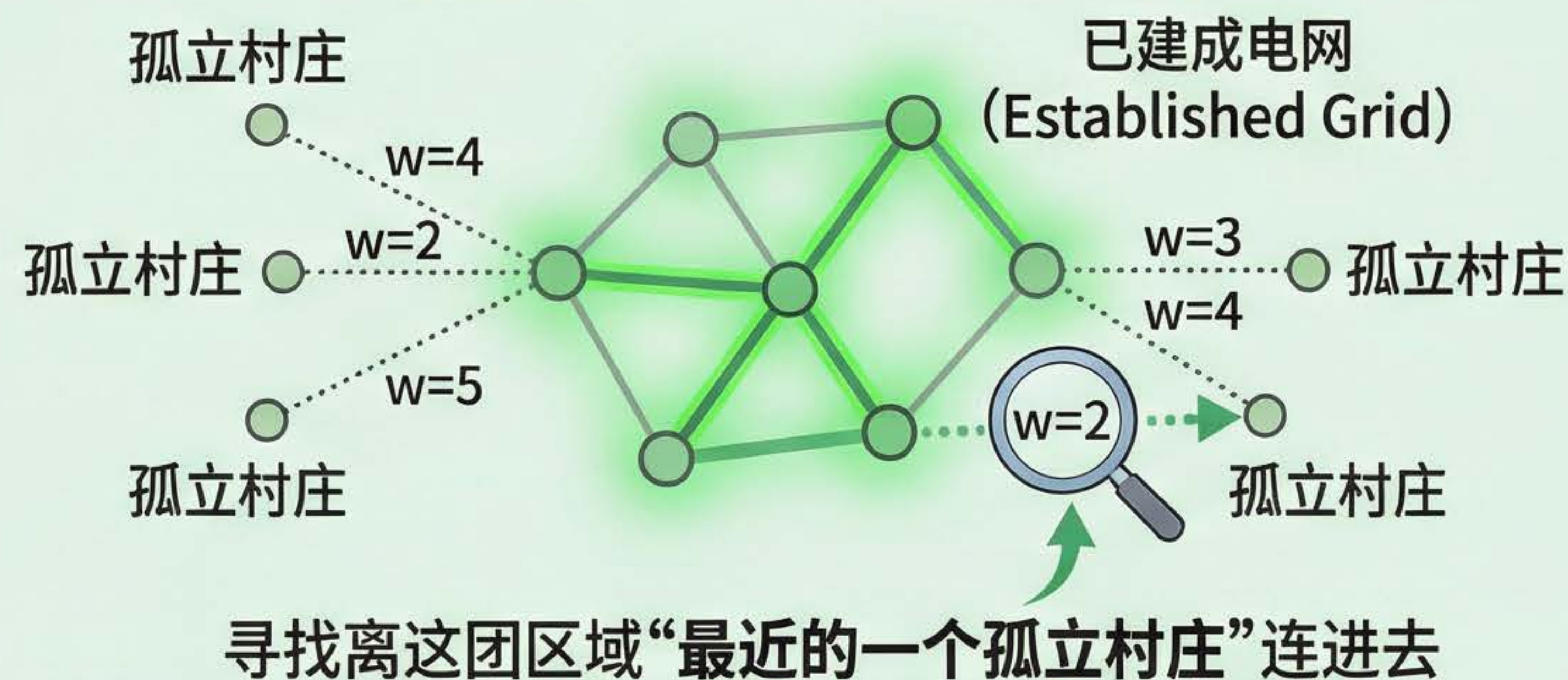
$$d[v] = \min(d[v], d[u] + w)$$

公式解读：关注“起点到当前点”的总代价。新来的边必须加上前面的历史成本。

应用场景：• 📍 GPS导航、• 🛵 外卖路线规划。

Prim（最小生成树 MST）

🤝 “极致的集体主义”



$$d[v] = \min(d[v], w)$$

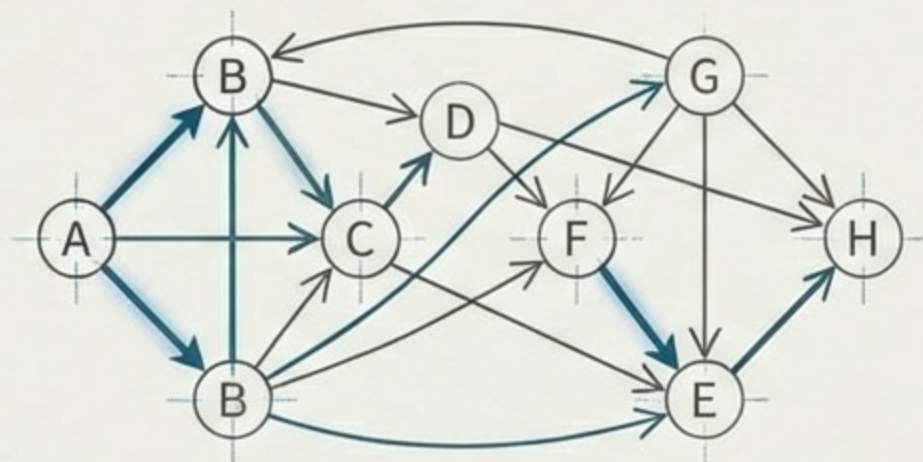
公式解读：只关注“新节点到已建成树”的距离。不关心它离最初的起点有多远（没有历史包袱）。

应用场景：• 🌐 铺设光缆、• 💻 电路板布线。

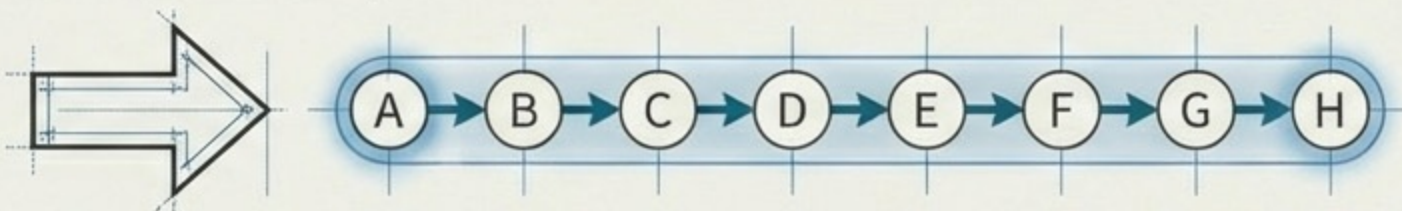
拓扑排序：DAG的线性化

存在拓扑排序 $\Leftrightarrow$ 图是DAG（无环有向图）

Before



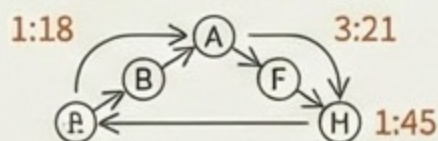
Linearization $\Rightarrow$



After

DFS

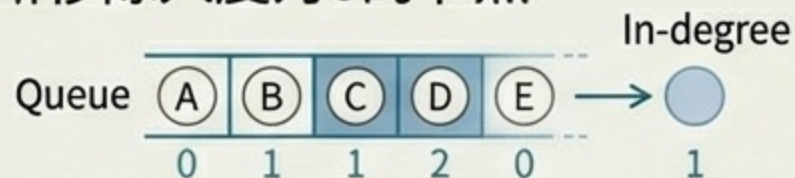
- 逆向思维：按完成时间 $f[v]$ 倒序排列



$O(V+E)$

Kahn (BFS)

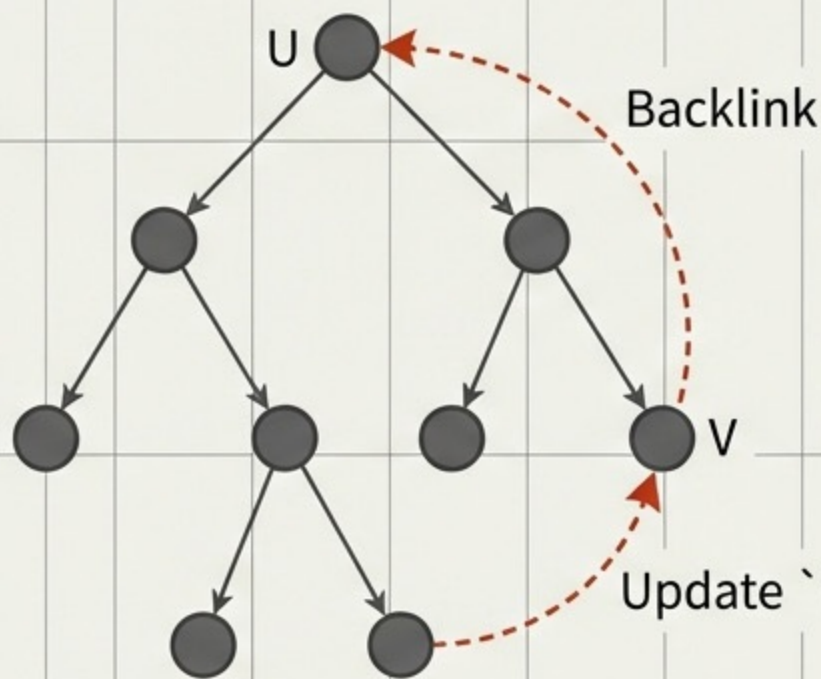
- 不断移除入度为0的节点



$O(V+E)$

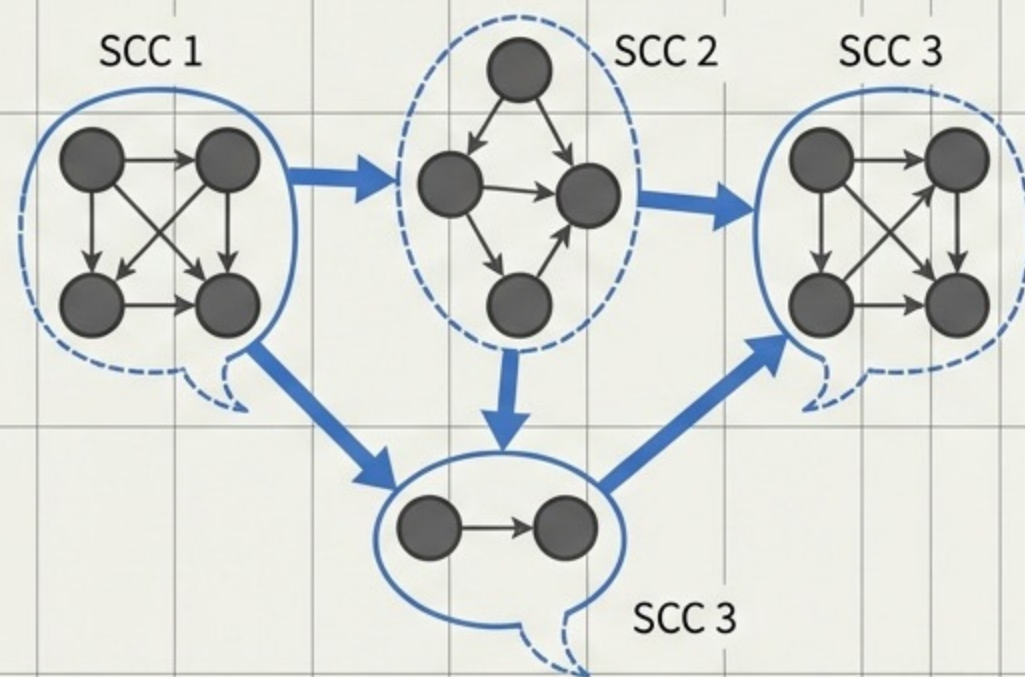
强连通分量 (SCC) : 深层结构的剖析

Tarjan (DFS)

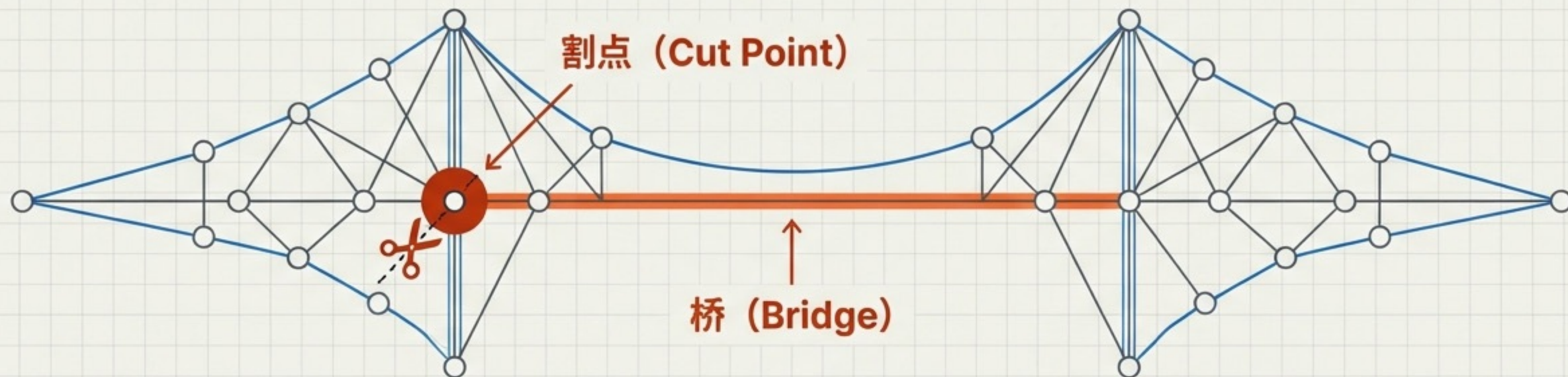


dfn: 时间戳 | low: 追溯值

If $\text{low}[v] == \text{dfn}[v]$:
Found SCC Root



连通性分析：寻找脆弱点



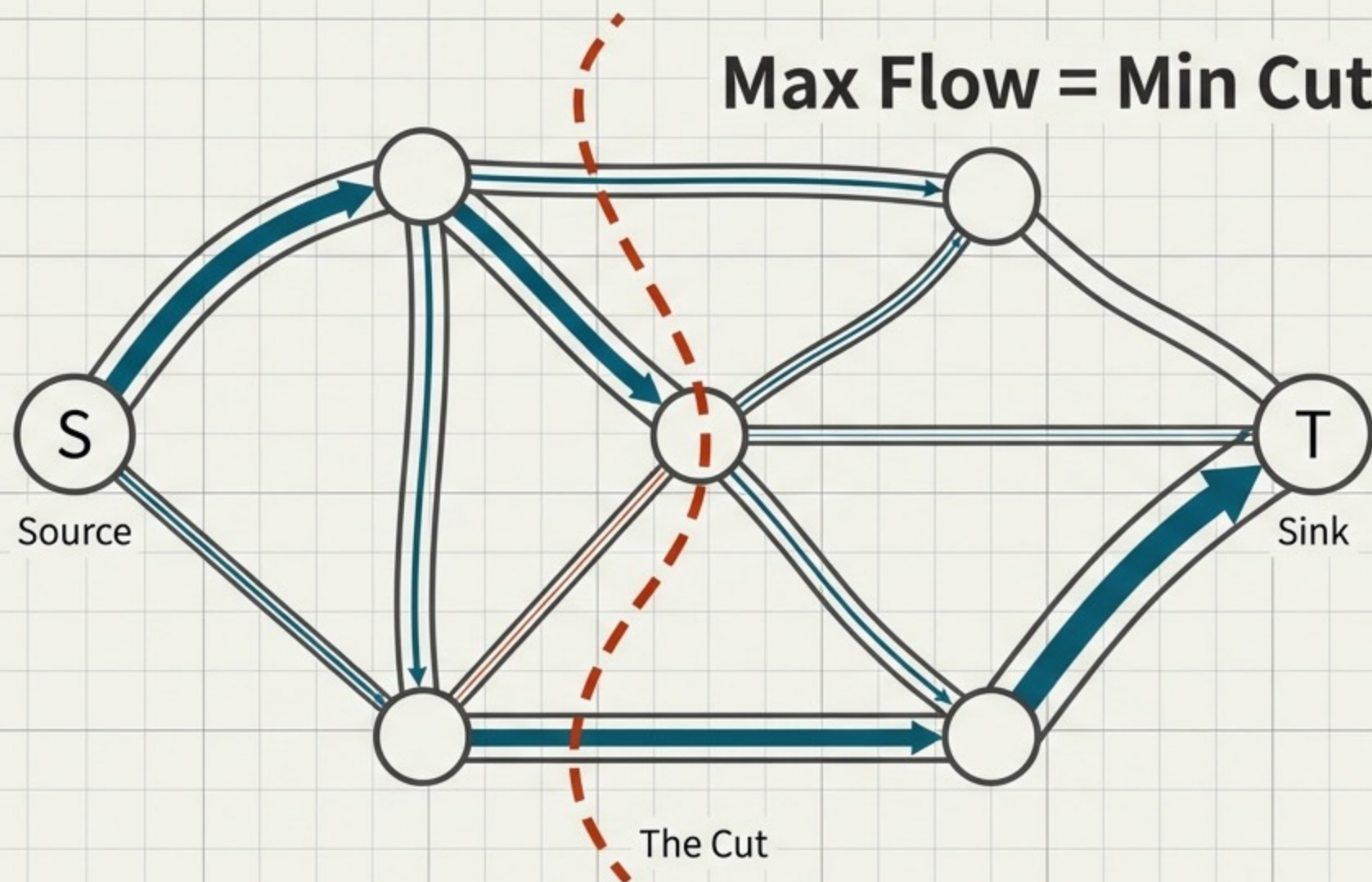
If this breaks, the network splits.

技术区分：Tarjan变体

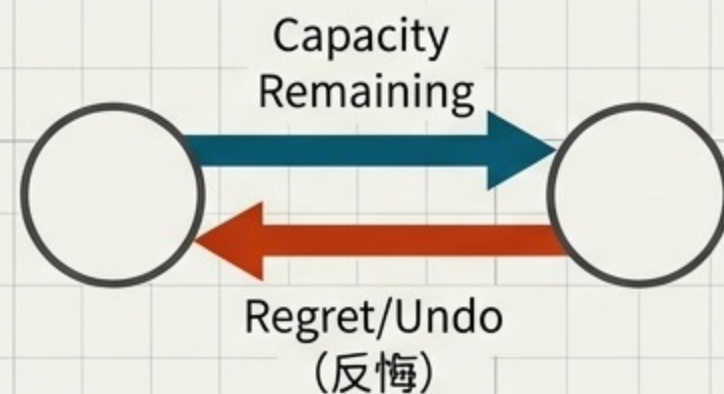
- **桥 (Bridge)** : Strict Inequality: $\text{low}[v] > \text{dfn}[u]$ (Child has NO way back)
- **割点 (Cut Point)** : Inclusive Inequality: $\text{low}[v] \geq \text{dfn}[u]$ (Child can only reach parent)

网络流：最大流最小割定理

Max Flow = Min Cut



Residual Graph (残余图)



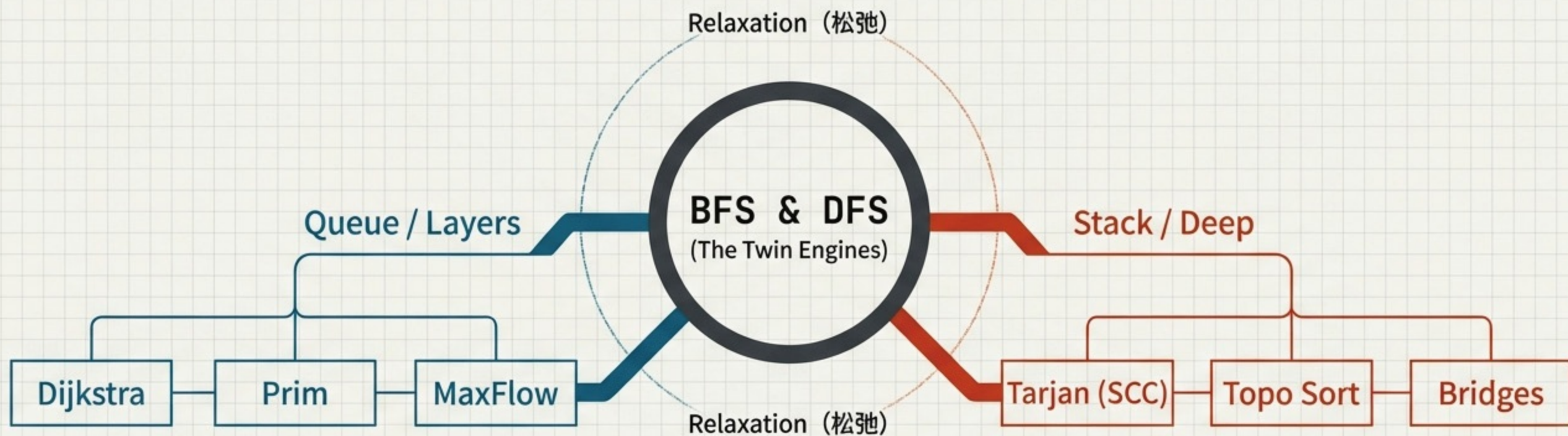
Algorithms

- Ford-Fulkerson
- Edmonds-Karp (BFS)

全景速查：复杂度与核心结构

算法 (Algorithm)	时间复杂度 (Time)	核心结构 (Core Structure)
BFS / DFS	$O(n + m)$	Queue / Stack
Dijkstra	$O(m \log n)$	Min-Heap
Bellman-Ford	$O(nm)$	Edge Relaxation
Floyd-Warshall	$O(n^3)$	DP Matrix
Kruskal / Prim	$O(m \log n)$	Union-Find / Min-Heap
Tarjan SCC	$O(n + m)$	DFS + Stack
Edmonds-Karp	$O(nm^2)$	BFS + Residual Graph

主线地图：回归本质



图论算法的关键不在于背诵代码，而在于知道哪种遍历最契合当下的问题。

算法落地：城市交通网的“X光透视”

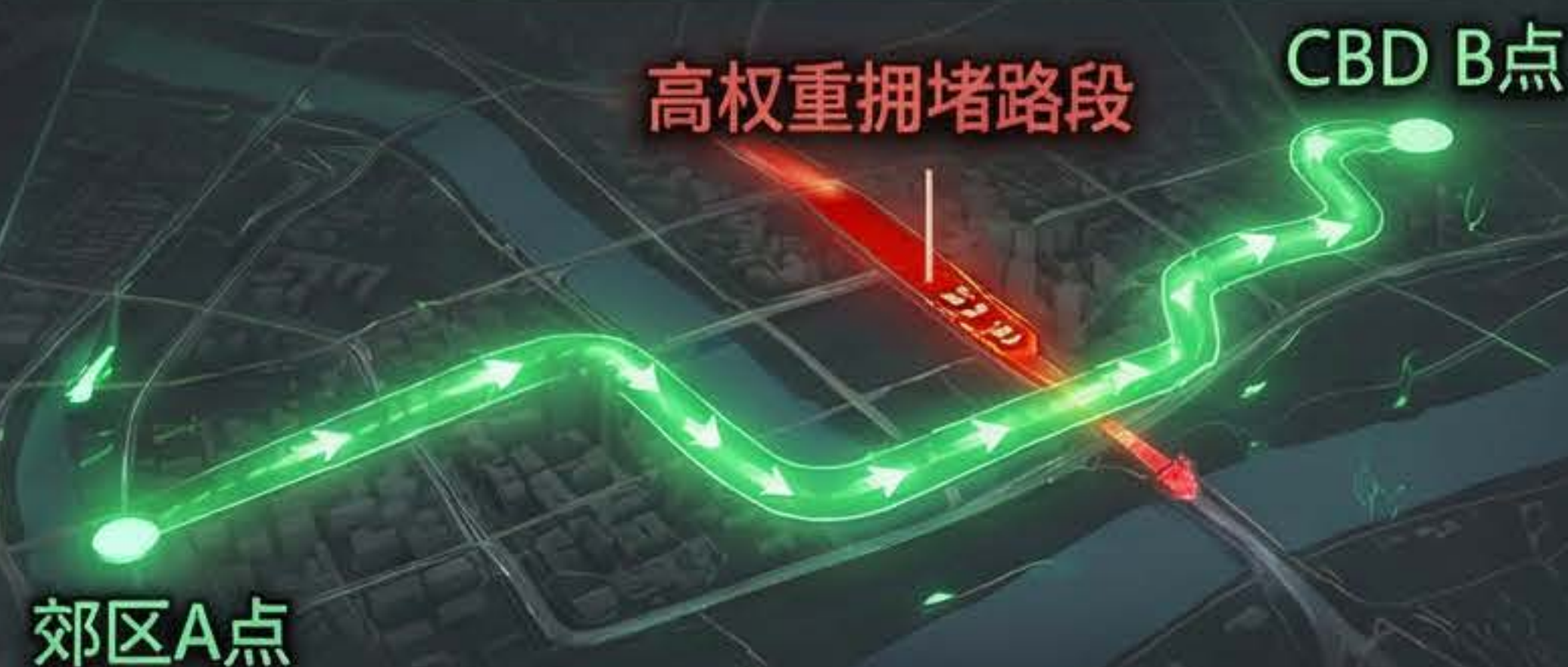
当我们把所有图论算法叠加在同一座城市上

透视 1: [最小生成树 MST] Kruskal 算法



“全局最低成本连通策略”

透视 2: [最短路径] Dijkstra / A* 算法



“避开拥堵，动态计算个人最优耗时”

透视 3: [最大流最小割] 网络流算法



“精准定位导致全城堵车的核心咽喉路段”

透视 4: [强连通分量 SCC] Tarjan 算法



“即使全是单行道，在这个圈内你也永远能绕回原地”

“图论算法不是一行行干瘪的代码，而是我们解构、量化、并优化这个复杂现实世界的终极透镜。”



图论算法核心讲义

结构决定性质，遍历发现真理。

Structure determines properties; traversal reveals truth.